PROGRAMAÇÃO FUNCIONAL

REDUÇÃO DE ESTRUTURAS COM FOLDR E FOLDL

CENTRO FEDERAL DE EDUCAÇÃO
TECNOLÓGICA DE MINAS GERAIS

MARCOS.EMILIANO@CEFETMG.BR
MARCOS EMILIANO

OBJETIVOS DA AULA

- Compreender o conceito de **folding** como a generalização da **indução** sobre listas.
- Analisar o comportamento do **foldr** (redução à direita) e do **foldl** (redução à esquerda).
- Visualizar a **árvore** de expansão e a ordem de **avaliação** das duas abordagens.

MOTIVAÇÃO

A maioria das funções **recursivas** sobre **listas** segue um **padrão** de projeto muito **previsível**:

1. Um **caso base** que define o resultado para a **lista vazia** ([]).
2. Um **caso recursivo** que combina a **cabeça** da lista (**x**) com o **resultado** da chamada recursiva na **cauda** (**xs**).

MOTIVAÇÃO

1. sum [] = 0
2. sum (x : xs) = x + sum xs

1. concat [] = []
2. concat (xs : xss) = xs ++ concat xss

1. and [] = True
2. and (x : xs) = x && and xs

MOTIVAÇÃO

Em vez de escrevermos a recursão **manualmente** repetidas vezes, **extraímos** esse padrão para funções de **ordem superior** chamadas **folds**.

MOTIVAÇÃO

Um **fold** "dobra" ou "reduz" uma **estrutura de dados** a um **único valor** (que pode ser um **primitivo**, uma **nova lista**, ou até uma **árvore**), utilizando uma **função** de **combinação** e um **valor inicial** (acumulador).

FOLDR (FOLD RIGHT) - REDUÇÃO PELA DIREITA

Assinatura:

```
foldr :: (a -> b -> b) -> b -> [a] -> b
```

- O **foldr** processa a lista da **direita** para a **esquerda** conceitualmente.

Visão Estrutural: O foldr substitui o construtor cons (**:**) pela **função fornecida** e a **lista vazia** ([]) pelo valor **acumulador**.

FOLDR (FOLD RIGHT) - REDUÇÃO PELA DIREITA

1. foldr :: (a -> b -> b) -> b -> [a] -> b
2. foldr _ v [] = v
3. foldr f v (x : xs) = x `f` foldr f v xs

FOLDR (FOLD RIGHT) - REDUÇÃO PELA DIREITA

```
-- Implementando a soma de todos os elementos
somaLista :: [Int] -> Int
somaLista = foldr (+) 0
concat = foldr (++) []
and = foldr (&&) True
-- O que acontece na prática ao avaliar:
--    foldr (+) 0 [1, 2, 3]
-- Estrutura original: 1 : (2 : (3 : []))
-- Substituição:       1 + (2 + (3 + 0))
-- Resultado:          6
```

FOLDR (FOLD RIGHT) - REDUÇÃO PELA DIREITA

```
foldr (+) 0 (x : y : z : [])   =
x + foldr (+) 0 (y : z : [])   =
x + (y + foldr (+) 0 (z : [])) =
x + (y + (z + foldr (+) 0 [])) =
x + (y + (z + 0))
```

FOLDL (FOLD LEFT) - REDUÇÃO PELA ESQUERDA

Assinatura:

```
foldl :: (b -> a -> b) -> b -> [a] -> b
```

- O **foldl** processa a lista da **esquerda** para a **direita**, **atualizando** um **acumulador** a cada passo.

Visão Estrutural: É análogo a um laço de repetição (while/for) em linguagens imperativas, onde o estado do acumulador é passado iterativamente.

FOLDL (FOLD LEFT) - REDUÇÃO PELA ESQUERDA

```
1. foldl :: (b -> a -> b) -> b -> [a] -> b
2. foldl _ v []     = v
3. foldl f v (x : xs) = foldl f (f v x) xs
```

FOLDL (FOLD LEFT) - REDUÇÃO PELA ESQUERDA

```
-- Soma usando foldl
-- Avaliação: (((0 + 1) + 2) + 3)
somaListaEsq :: [Int] -> Int
somaListaEsq = foldl (+) 0
```

FOLDL (FOLD LEFT) - REDUÇÃO PELA ESQUERDA

```
foldl (+) 0 (x : (y : (z : []))) =
foldl (+) (0 + x) (y : (z : [])) =
foldl (+) ((0 + x) + y) (z : []) =
foldl (+) (((0 + x) + y) + z) [] =
((0 + x) + y) + z
```

FOLDL (FOLD LEFT) - REDUÇÃO PELA ESQUERDA

```
-- Inversão de lista (padrão clássico do foldl)
inverteLista :: [a] -> [a]
inverteLista = foldl (\acc x -> x : acc) []
-- Passo a passo para [1, 2, 3]:
-- 1. acc = [], x = 1        -> [1]
-- 2. acc = [1], x = 2       -> [2, 1]
-- 3. acc = [2, 1], x = 3    -> [3, 2, 1]
```

FOLDR VS FOLDL

Para a adição, o resultado é o mesmo...

```
> foldr (+) 0 [1,2,3]
6
> foldl (+) 0 [1,2,3]
6
```

FOLDR VS FOLDL

Porém, para a subtração o resultado é diferente...

```
> foldr (-) 0 [1,2,3]
2
> foldl (-) 0 [1,2,3]
-6
```

FOLDR VS FOLDL

Visualizando o que o Haskell fez por debaixo dos panos:
Para o **foldr** (Agrupamento à direita):

- Passo 1: Substitui os construtores da lista 1 : (2 : (3 : []))
- Passo 2: 1 - (2 - (3 - 0))
- Passo 3: 1 - (2 - 3)
- Passo 4: 1 - (-1)

Resultado Final: 2

FOLDR VS FOLDL

Visualizando o que o Haskell fez por debaixo dos panos:
Para o **foldl** (Agrupamento à esquerda / Acc):

- Passo 1: Pega o zero inicial e aplica no primeiro elemento.
- Passo 2: (((0 - 1) - 2) - 3)
- Passo 3: ((-1 - 2) - 3)
- Passo 4: (-3 - 3)
- Resultado Final: -6

COMPARATIVO VISUAL

Associatividade:

- foldr f z [1,2,3] -> f 1 (f 2 (f 3 z))
  (Associatividade à direita).

- foldl f z [1,2,3] → f (f (f z 1) 2) 3
  (Associatividade à esquerda).

COMPARATIVO VISUAL

Listas Infinitas e Curto-circuito:

- O **foldr** funciona perfeitamente com **listas infinitas** se a função **combinadora** não avaliar **estritamente** o seu segundo argumento (ex: operador lógico || ou &&).
- O **foldl** nunca funciona com listas infinitas, pois precisa **percorrer** toda a estrutura até o **final** para retornar o **acumulador**.

COMPARATIVO VISUAL

Aviso:
O foldl padrão acumula **expressões não avaliadas** (thunks), o que pode causar **StackOverflow**. Em código de produção, utiliza-se sempre o **foldl'** (restrito/estrito) importado de Data.List.

EXERCICIO

Questão 1: **Contagem Universal**
Utilizando estritamente a função **foldr** (sem utilizar recursão explícita ou a função nativa length), implemente uma função contaElementos :: [a] -> Int que retorne o **tamanho** de uma **lista** qualquer.
Dica: Na função lambda do fold, o valor do **elemento atual** da lista importa para o cálculo do comprimento?

EXERCICIO

Questão 2: Achatamento de Estruturas
Implemente a função **achataLista** :: [[a]] -> [a] utilizando foldr. A função deve receber uma **lista de listas** e concatená-las em uma **única lista plana**. Exemplo esperado: achataLista [[1,2], [3,4], [5]] deve retornar [1,2,3,4,5].
Dica: Utilize o operador de concatenação de listas ++.

EXERCICIO

Questão 3: Contagem Condicional com Acumulador
Utilizando **foldl**, implemente a função **contaOcorrencias** :: Eq a => a -> [a] -> Int. A função recebe um **elemento alvo** e uma **lista**, e retorna **quantas vezes** o elemento **alvo aparece** na **lista**.
Exemplo: contaOcorrencias 'a' "matematica" deve retornar 3.

EXERCICIO

Questão 4: Análise Teórica de Associatividade

Sem executar o código no GHCi, analise as duas expressões abaixo e explique o motivo de produzirem resultados fundamentalmente diferentes. Qual é o resultado de cada uma?

1. foldr (-) 0 [2, 4, 6]
2. foldl (-) 0 [2, 4, 6]

EXERCICIO

Questão 5: Avaliação Preguiçosa e Listas Infinitas
Considere uma **lista infinita** de valores **booleanos** gerada por (False : repeat True), ou seja, [False, True, True, True...]. Se avaliarmos **foldr** (&&) True (False : repeat True), o GHCi retorna False instantaneamente. Porém, se avaliarmos **foldl** (&&) True (False : repeat True), o GHCi entra em um **laço infinito** e **trava**.

Continua...
---->

EXERCICIO

Questão 5: Avaliação Preguiçosa e Listas Infinitas

Explique, a partir da avaliação preguiçosa (lazy evaluation) do Haskell e da **ordem de expansão** dos **folds**, por que o **foldr** consegue **terminar** o **cálculo** mesmo a **lista sendo infinita**, enquanto o **foldl falha**.

DÚVIDAS???